Programowanie gier komputerowych

Analiza kodu GTA 3

inż. Mateusz Połeć

Semestr zimowy 2026/2027

Wprowadzenie

Rok 2001, końcówka października, firma Rockstar wydaje gre Grand Theft Auto III. Tytuł początkowo debiutuje wyłącznie na konsole Play Station 2, dopiero rok później na komputery osobiste i dwa lata później na platformę Xbox.
Żeby zobrazować sobie rozmiar przedwsięwzięcia, weźmy pod uwagę specyfikację sprzętu konsoli PS2 i fakt, że gra (jak na tamte lata) dosć dobrze radzi sobie z wyzwaniem, jakim jest stworzenie 'immersyjnego' open-world tytułu.
Kod, który jest wynikiem inżynierii wstecznej, dostępny jest w repozytorium: `https://github.com/hottabxp/re3`

PS2 – Specyfikacja

Konsola od Sony dysponowała następującymi podzespołami:

- Procesor (CPU) - 64-bitowy, taktowanie 290-300 MHz (reklamowany jako konsola 128 bitowa, przez jednostki wektorowe do obliczeń graficznych).
- Pamięć RAM (32 MB RDRAM), przepustowość 3,2 GB/s.
- Procesor graficzny (GPU) - "Graphics Synthesizer", taktowanie 147 MHz.
- Nośniki danych: DVD-ROM/CD-ROM, oraz karty pamięci (standardowo do 8MB).

PS2/GTA III – Odczyt z płyty

Świat gry i wszystkie informacje (świat, modele, tekstury, dźwięki) dla GTA III były odczytywane prosto z płyty do pamięci RAM. Prędkość odczytu dla płyt CD-ROM w PS2 jest równy około 3.6-3.7 MB/S, a dla płyt DVD około 5.3-5.5 MB/S. Znaczy to, że Rockstar miał do rozwiązania istotny problem "wąskiego gardła"(bottle-neck) dla swojego produktu.
W tym celu firma ograniczyła koszt ruchu głowicy, organizując odczyt według fizycznych offsetów plików na nośniku. Kod wybiera kolejny oczekujący plik znajdujący się na płycie, a po dojściu zawija kolejkę do początku. Dodatkowo, sąsiadujące pliki są zgrupowane w jeden transfer (w tym przypadku maksymalnie cztery). Jest to idea algorytmu C-SCAN, który pozwala na ruch głowicy tylko w jednym kierunku, a po dojściu na koniec nośnika danych zaczyna od początku.
Konkretną implementację można znaleźć w pliku `Streaming.h` i `Streaming.cpp`

PS2/GTA III – Zarządzanie pamięcią

System zarządzania pamięcią oparty jest na idei `Pool Allocatora`, a konkretniej `fixed-size slot pool allocator`. Cała pamięć na dany typ obiektów (Pojazdy, budynki, humanoidy) jest alokowana raz, na starcie gry. Kod nigdy nie nie woła `new/delete`, jedynie umieszcza (placement) dane, które powinny trafić na stere, w gotowym buforze.
Rozmiary puli są z góry znane i dostępne w pliku `config.h`. Całkowita pamięć konkretnej puli to `liczba slotów * sizeof(BiggestType)`.

```
enum Config
{
    ...
        NUMPEDS = 140,          // 90 on PS2
        NUMVEHICLES = 110,      // 70 on PS2
        NUMBUILDINGS = 5500,    // 4915 on PS2
        NUMDUMMIES = 2802, // 2368 on PS2
    ...
}
```

PS2/GTA III – Zarządzanie pamięcią

Przykładowe pule obiektów:

```
typedef CPool<CPed,CPlayerPed> CPedPool;
typedef CPool<CVehicle,CAutomobile> CVehiclePool;
```

Alokacja buforów (jeden new[] na całą pulę). `entires` to surowy bufor bajtów, a nie tablica obiektów!

```
CPool(int32 size)
{
    m_entries = (U*)new uint8[sizeof(U)*size];
    m_flags = new uint8[size];
    m_size = size;
    ...
}
```

Kod związany z zarządzaniem pamięcią znajduje się w plikach `templates.h`, `Pools.h` i `Pools.cpp`.

PS2/GTA III – Zarządzanie pamięcią

Alokacja i zwalnanie slotu w buforze bez `new/delete`. `allocPtr` pamięta ostatnią pozycję, więc kolejne przeciążone `New` zwykle znajduje wolny slot w złożoności obliczeniowej `O(1)`, nie przeszukując puli za każdym razem.

```
T *New(void){
    bool wrapped = false;
    do
        if(++m_allocPtr == m_size){
            if(wrapped) return nil;
            wrapped = true;
            m_allocPtr = 0;
        }
    while(!GetIsFree(m_allocPtr));
    SetIsFree(m_allocPtr, false);
    SetId(m_allocPtr, GetId(m_allocPtr)+1);
    return (T*)&m_entries[m_allocPtr];
}
```

PS2/GTA III – Zarządzanie pamięcią

Jak uniknięto w takim razie niebezpieczeństwa, jakim jest dangling pointer?
Gra używa 32-bitowego uchwytu (handle):

```
#define POOLFLAG_ID      0x7f    // licznik generacji (7 bit)
#define POOLFLAG_ISFREE  0x80    // czy slot wolny (1 bit)
```

Jeśli obiekt pod danym indeksem został skasowany i slot został ponownie użyty przez inny obiekt, jego ID się zmieniło – stary handle przestaje pasować i GetAt() zwraca nil zamiast wskaźnika na "martwy"/nadpisany obiekt. To dokładnie rozwiązuje klasyczny problem ABA lub dangling pointer bez potrzeby garbage collectora czy weakptr, kosztem zaledwie 1 bajta metadanych na slot.

```
T *GetAt(int handle){
    return m_flags[handle>>8] == (handle & 0xFF) ?
           (T*)&m_entries[handle >> 8] : nil;
```

PS2 / GTA III – Obsługa cheat codes

Cała obsługa cheat code'ów znajduje się w pliku `Pad.cpp`, gdzie zależnie od wciśniętego przycisku kod porównuje wprowadzony ciąg znaków:

```
void CPad::DoCheats(int16 unk)
{
#ifdef GTA_PS2_STUFF
        if ( GetTriangleJustDown() )
                AddToCheatString('T');

        if ( GetCircleJustDown() )
                AddToCheatString('C');

    ...
#endif
}
```

PS2 / GTA III – Obsługa cheat codes

```
void CPad::AddToPCCheatString(char c)
{
        for ( int32 i = ARRAY_SIZE(KeyBoardCheatString
            ) - 2; i >= 0; i-- )
                KeyBoardCheatString[i + 1] =
                    KeyBoardCheatString[i];

        KeyBoardCheatString[0] = c;

        #define _CHEATCMP(str) strncmp(str,
            KeyBoardCheatString, sizeof(str)-1)

        // "GUNSGUNSGUNS"
        if ( !_CHEATCMP("SNUGSNUGSNUG") )
        WeaponCheat();
    ...
}
```

PS 2 / GTA III – Działanie broni

Kod odnośnie broni jest zawarty również w jednej, monolitycznej klasie, dostępnej w pliku `Weapon.cpp` i `Weapon.h`. Bronie działają na zasadzie maszyny stanów, przechodząc przez stany:

- READY
- FIRING
- RELOADING
- OUT OF AMMO
- MELEE MADE CONTACT

Bronie oczywiście dzielą się na palne i białe. Palne mają dodatkową logikę, obsługującą informację o pocisku:

- Hitscan: Wysyła raytrace i sprawia, że pocisk trafia w tej samej klatce. Brońmi, które wykorzystują te technike są m.in. Colt45, UZI, M16, Shotgun.
- Projectile: Tworzy fizyczny obiekt, na który działa fizyka. Brońmi, wykorzstującymi te technike są: Sniper Rifle, Rocket Launcher, Molotov, Granat.

PS 2 / GTA III – Działanie broni

Dla pewnych broni, a konkretnie dla `M16` i `HELICANNON` gra dodatkowo wprowadza dwie logiki strzelania:

```
fired = FireM16_1stPerson(shooter);
// lub
fired = FireInstantHit(shooter, source);
```

Różnica polega na tym, że `FireM161stPerson` liczy strzał wprost z kamery, czyli dosłownie z 'widoku lufy'. Strzał idzie dokładnie w tym kierunku, w którym patrzy kamera. Inaczej mierzenie przez celownik/lunetę, byłoby niespójne z tym, co gracz widzi na ekranie. Dokładnie ten sam warunek pojawia się przy wystrzałach z snajperki.

PS 2 / GTA III – Działanie broni

I ostatnia informacja odnośnie broni, jako ciekawostka, lub bug... Rockstar w kodzie obsługującym bronie białe, celowo zwiększył obrażenia z kija bejsbolowego dwukrotnie, kiedy ofiarą ataku jest gracz:

```
if ( victimPed->IsPlayer() && isBat ) // wtf, it's not
     fair
     victimPed->InflictDamage(shooterPed, m_eWeaponType
         , 2.0f*info->m_nDamage, PEDPIECE_TORSO,
         localDir);
else
     victimPed->InflictDamage(shooterPed, m_eWeaponType
         ,       info->m_nDamage, PEDPIECE_TORSO,
         localDir);
```

PS 2 / GTA III – Działanie świata

Gra używa stałego podziełu przestrzeni na siatkę sektorów. Obszar świata obejmuje współrzędne:

$$x, y \in (-2000, 2000)$$

podzielony jest na 100x100 sektorów, a każdy sektor ma wielkość 40x40.

```
#define SECTOR_SIZE_X (40.0f)
#define SECTOR_SIZE_Y (40.0f)

#define NUMSECTORS_X (100)
#define NUMSECTORS_Y (100)

#define WORLD_SIZE_X (NUMSECTORS_X * SECTOR_SIZE_X)
#define WORLD_SIZE_Y (NUMSECTORS_Y * SECTOR_SIZE_Y)

#define WORLD_MIN_X (-2000.0f)
#define WORLD_MIN_Y (-2000.0f)
```

Omawiamy kod dostępny jest w plikach `World.h` i `World.cpp`.

PS 2 / GTA III – Działanie świata

Każdy z sektorów ma osobne listy wskaźników na obiekty różnych typów – budynki, pojazdy, pieszych. Dla każdego typu istnieje też lista `OVERLAP` przeznaczona dla encji, których obszar może obejmować więcej, niż jeden sektor. Dzięki temu algorytmy przetwarzania przestrzennego unikają iteracji po całym świecie.
Dla przykładu:

- `FindObjectsInrange` - Oblicza prostokąt sektorów pokrywający pokrywający zadany promień/bryłę.
- `ProcessLineOfSight` - Przechodzi wyłącznie przez sektory przecinane przez promień
- `ProcessVerticalLine` - Przechodzi przez pojedynczy sektor (promień pionowy, nie zmienia wartości x ani y).

PS 2 / GTA III – Pojazdy (pociągi)

Pociągi w grze nie poruszają się po fizycznych szynach, ani nie korzystają z klasycznej "nawigacji", zamiast tego, całość porusza się po zamkniętej linii punktów, a jego pozycja jest stale aktualizowana matematycznie:

- Gra wczytuje liczbę punktów, a następnie współrzędne (x, y, z) każdego punktu.
- Punkty opisują jedną szynę, kod przesuwa je o `0.9` jednostki prostopadle do kierunku jazdy, aby otrzymać środek torów.
- Dla każdego punktu obliczana jest odległość `t` od początku trasy. Jest to odległość w płaszczyźnie XY.
- Wysokość toru jest korygowana przez pionowy test kolizji z mapą. Dzięki temu pociąg dostosowuje się do geometrii świata.

Kod dostępny jest w klasie `Train.cpp` i `Train.h`

PS 2 / GTA III – System powtórek

W grze dostępny był system, który umożliwiał odegranie kilku ostatnich sekund rozgrywki. Nie działało to na zasadzie nagrywania obrazu ani ponownym wykonaniem zapisanych ruchów gracza. Gra w każdej klatce zapisuje niewielki zestaw danych o kamerze, pieszych, pojazdach, czasie, pogodzie i śladach pocisków, a potem odtwarza z nich scenę. Kod dostępny w `Replay.h` i `Replay.cpp`.

- `CReplay::Update` co klatkę (o ile nie trwa pauza lub cut-scene) dopisuje stan poszczególnych systemów (pojazdy, piesi) do bufora.
- Bufor składa się z ośmiu mniejszych puli pamięci, każda po 100 000 bajtów. Czyli zajmuje około 0,8 MB.
- Bufor działa cyklicznie – po zapełnieniu kolejny bufor zastępuje najstarszy zapisany. Długość replay zależy więc od tego, ile się działo wokół gracza w konkretnym momencie, a nie jest stałą liczbą sekund.
- Po uruchomieniu replay, gra zachowuje bieżący stan świata, opróżnia pule pieszych i pojazdów, ładuje modele potrzebne do powtórki i odtwarza ich stan z zapisanego bufora.

PS 2 / GTA III – Easter egg

Rockstar lubił zostawiać dla graczy niespodzianki, jedną z takich niespodzianek jest możliwość zmiany wielkości księżyca. W klasie `CWeapon`, w funkcji `CWeapon::FireSniper` znajdziemy taki zapis:

```
CVector source = cam->Source;
CVector dir    = cam->Front;

if ( DotProduct(dir, CVector(0.0f, -0.9894f, 0.145f))
    > 0.997f )
     CCoronas::bSmallMoon = !CCoronas::bSmallMoon;
```

Natomiast w kodzie rysowania księżyca `Clouds.cpp`:

```
if(CCoronas::bSmallMoon){
     szx *= 4.0f;
     szy *= 4.0f;
}else{
     szx *= 10.0f;
     szy *= 10.0f;
}
```

PS2 / GTA III – Easter egg

- Kamera snajperki ma znany kierunek celowania `cam->Front`
- Gra sprawdza iloczyn skalarny pomiędzy kierunkiem lufy, a stałym wektorem `(0.0f, -0.9894f, 0.145f)`
- Księżyc zawsze renderuje się w stałej pozycji względem kamery -> więc Rockstar idąc na łatwizne, określając stały wektor, względem którego księżyć będzie renderowany dla gracza.
- Jeżeli wynik iloczynku skalarnego przekracza próg `0.997f`, oznacza to, że gracz celuje dokładnie w księżyc (z małym marginesem błędu)
- I końcowo przełącza flagę `bSmallMoon`.

PS 2 / GTA III – Poziom zainteresowania policji

Produkcja zawiera również utrudnienie, jakim jest próba ukarania gracza pościgiem policyjnym za każde przestępstwo, jakie popełni. Działa to na zasadzie punktów chaosu, które przekładają się na poziom zainteresowania policjii (gwiazdki). Kod z plików `Wanted.h` i `Wanted.cpp`

Chaos	Poziom	NPC	Radiowozy	Blokady drogowe
0–39	0	0	0	0
40–199	1	1	1	0
200–399	2	3	2	0
400–799	3	4	2	4
800–1599	4	6	2	8
1600–3199	5	8	3	10
3200+	6	10	3	12

Tabela: Zależność poziomu chaosu od dostępnych jednostek policji.

PS 2 / GTA III – Poziom zainteresowania policji

Wartość przestępstwa (np. potrącenie pieszego bazowo daje 18 punktów, postrzelenie pieszego 30, policjanta 80, zniszczenie helikoptera 400...) są mnożone przez współczynnik uwzględniający strefę, czułość i tryb gry. System skaluje również różne jednostki zaangażowane w pościg:

- 1-2 gwiazdki: Podstawowe siły policji i rosnące limity patroli.
- 3 gwiazdki: Wymagany jeden helikopter.
- 4 gwiazdki: SWAT.
- 5 gwiazdek: FBI; dwa helikoptery.
- 6 gwiazdek: Wojsko.

PS2 / GTA III – Bugi pozostawione przez Rockstar

Pierwszy ciekawy błąd znajduje się w kodzie `Ped.cpp`, w metodzie `CPed::SetEvasiveDive`. Logika zawarta w tej funkcji odpowiadała za unikanie przez NPC kolizji z nadjeżdżającym samochodem. Problem w tym, że NPC zamiast odskakiwać od pojazdu, wskakiwały wprost pod niego...

PS2 / GTA III – Bugi pozostawione przez Rockstar

Kod zbugowany:

```
angleToFace = GetRadianAngleBetweenPoints(carPos,
    pedPos);
...
m_fRotationCur = angleToFace;
SetPedState(PED_DIVE_AWAY);
```

Pieszy ustawia się przodem do auta, a potem skakał dokładnie w tym kierunku.

```
angleToFace += PI;
dangerZone = LimitRadianAngle(angleToFace -
    vehDirection);
angleToFace = dangerZone > 0 ? vehDirection + HALFPI :
     vehDirection - HALFPI;
```

Poprawka: najpierw odwraca kierunek od auta, a następnie wybiera, czy odskoczyć w lewo, czy prawo.

PS2 / GTA III – Bugi pozostawione przez Rockstar

Skanowanie przy krawędzi mapy mogło wyjść poza tablicę sektorów. Oryginał dopuszcza indeks 100, choć indeksy sektorów ograniczone są do 0-99. Grozi to odczytem poza tablicą i crashem:

```
#ifdef FIX_BUGS
        if(maxX >= NUMSECTORS_X) maxX = NUMSECTORS_X -
            1;
#else
        if(maxX >= NUMSECTORS_X) maxX = NUMSECTORS_X;
#endif
```

PS2 / GTA III – Cenzura gry

W wersji francuskiej i niemieckiej gra ustawia flagę `nastyGame = false` i ogranicza brutalne treści, m.in. krew przy trafieniu, atakowanie leżących postaci. Wersje wydane na te rynki dostają też osobny wariant logo w menu.